

第 9 章 反向传播与自动微分——从链式法则到学习影响

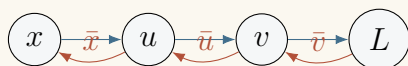
本章目标

反向传播不是“神经网络专属求导口诀”，而是 **reverse-mode automatic differentiation**：在计算图上系统复用链式法则，使“一个标量损失对海量参数求梯度”的成本与少数次前向计算同阶。本章将从 Jacobian、JVP 与 VJP 建立统一形式，手推线性层、激活与 softmax 的反传，解释为什么不应显式构造巨大 Jacobian；随后讨论 gradient check、Hessian-vector product、activation checkpointing，以及梯度内积如何预测 finetuning 的跨样本影响。最后用 ICLR 2025 Outstanding Paper 与 ICLR 2026 Oral 展示：梯度不仅能用来训练，也能成为解释 LLM 学习机制的研究对象。

Prerequisite Recall | 先把这些知识唤醒

- 熟练使用标量链式法则。
- 会区分 Jacobian、gradient 与 matrix transpose。
- 记住反向传播是计算梯度的算法，不是新的微积分定理。

Visual Derivation | 先看结构，再读公式



蓝色是数值的 forward flow，红色是 cotangent 的 reverse flow；不要把“图上方向反了”误解成函数本身被逆运算。

Misconception Check | 本章最容易掉进去的坑

“反向传播需要显式构造每层 Jacobian”是错误的。现代 autodiff 的关键正是只计算 Jacobian 与向量的乘积 (VJP/JVP)，避免构造巨大的 Jacobian。

Concept Contrast | 把相邻概念主动分开

	Forward mode (JVP)	Reverse mode (VJP)
传播	输入切向量	输出余切向量
适合	少输入、多输出	多参数、标量 loss
深网训练	通常不优	标准选择

Checkpoint | 开始正文前的 30 秒自测

一个模型有十亿参数但只有一个标量 loss，为什么 reverse mode 特别合适？**检查答案：**一次反向 sweep 可同时得到该标量对所有参数的梯度，而不必为每个参数做一次 forward derivative。

9.1 计算图：复杂函数只是局部函数的组合

一个程序可以写成有向无环图。例如

$$a = Wx, \quad h = \rho(a), \quad z = Uh, \quad L = \ell(z, y).$$

每个节点只知道局部运算，但全局导数由链式法则组合。若 $h = f(x)$ 、 $y = g(h)$ ，则

$$J_{y,x} = J_{y,h} J_{h,x}.$$

问题在于：真实模型的 Jacobian 可能大到根本不能显式存储。

9.2 JVP 与 VJP：我们真正需要的是 Jacobian 的“作用”

给定 $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$ ，Jacobian 为 $J \in \mathbb{R}^{m \times n}$ 。

Jacobian-vector product (JVP):

$$Jv,$$

描述输入沿方向 v 微小变化时，输出如何变化。

Vector-Jacobian product (VJP):

$$u^\top J,$$

描述输出上的 cotangent/adjoint 如何向输入反传。

forward-mode autodiff 自输入向输出传播 JVP；reverse-mode autodiff 自输出向输入传播 VJP。若输入参数数 n 极大而输出损失只有 1 个，reverse mode 特别合适。

命题 9.1 (标量损失的反向模式). 若最终输出 $L \in \mathbb{R}$, 令每个中间变量 v 的 *adjoint* 为

$$\bar{v} = \frac{\partial L}{\partial v},$$

则对局部关系 $v = f(u)$, 反传更新为

$$\bar{u} += \bar{v} J_{v,u}.$$

如果 u 被多个后继节点使用, 贡献必须求和。

9.3 两层网络的完整反向传播

设单样本

$$a_1 = W_1 x + b_1, \quad h = \rho(a_1), \quad z = W_2 h + b_2, \quad L = \ell(z, y).$$

令 $\delta_2 = \partial L / \partial z$ 。则

$$\begin{aligned} \frac{\partial L}{\partial W_2} &= \delta_2 h^\top, & \frac{\partial L}{\partial b_2} &= \delta_2, \\ \delta_1 &= (W_2^\top \delta_2) \odot \rho'(a_1), \\ \frac{\partial L}{\partial W_1} &= \delta_1 x^\top, & \frac{\partial L}{\partial b_1} &= \delta_1. \end{aligned}$$

权重梯度为什么总像“误差信号 $\times$ 输入”？

线性层 $z = Wx$ 的元素满足 $z_i = \sum_j W_{ij} x_j$ 。某个权重 W_{ij} 的变化只会按 x_j 的大小影响输出 z_i , 再乘上上游对 z_i 的敏感度 δ_i , 因此

$$\frac{\partial L}{\partial W_{ij}} = \delta_i x_j.$$

矩阵形式自然是外积 $\delta x^\top$ 。这个结构会在 Attention、LoRA、per-example gradients 和 influence 分析里反复出现。

9.4 Softmax + Cross-entropy 为什么通常合并反传？

若 $p = \text{softmax}(z)$, $L = -\log p_y$, 第 7 章已经得到

$$\frac{\partial L}{\partial z} = p - e_y.$$

框架通常不会先构造 softmax 的 $K \times K$ Jacobian 再乘交叉熵梯度, 而直接使用这个合并后的 VJP。高效 autodiff 的核心原则就是: 计算 Jacobian 的作用, 不构造 Jacobian 本身。

9.5 Reverse mode 的复杂度为什么可扩展？

设前向程序由 N 个基本标量/张量操作组成。在常见 smooth primitive 下，反向传播只需为每个操作计算一次局部 VJP，因此时间通常是前向的常数倍，而不是“参数数 $\times$ 一次前向”。代价转移到了内存：反向时需要知道前向产生的中间 activation。

这解释了一个工程事实：LLM 训练常常首先撞到显存，而不是纯 FLOPs。

9.6 Autodiff 不是符号微分，也不是有限差分

符号微分操作解析表达式，可能产生 expression swell。

有限差分近似

$$\frac{f(x+h) - f(x-h)}{2h},$$

存在截断误差与浮点消减误差。

自动微分则在程序执行轨迹上应用链式法则，在机器精度范围内得到导数。

Gradient check 中 h 不是越小越好

h 太大时 Taylor 截断误差明显； h 太小时 $f(x+h) - f(x-h)$ 会发生 catastrophic cancellation。实践中应扫描多个 h ，并用相对误差判断，而不是用一个极小步长宣布反传正确。

9.7 二阶信息不用显式 Hessian：Hessian-vector product

很多研究问题需要 Hv ，但 Hessian 是 $d \times d$ 。利用

$$Hv = \nabla_{\theta} (\nabla_{\theta} L(\theta)^{\top} v),$$

可以通过一次额外 autodiff 得到 HVP，而无需存储 H 。这使 curvature、influence function、Lanczos top eigenvalue 等分析在高维参数空间仍有可能。

9.8 梯度消失与爆炸：长 Jacobian 乘积的谱问题

若

$$h_{l+1} = F_l(h_l),$$

则

$$\frac{\partial L}{\partial h_l} = \frac{\partial L}{\partial h_L} J_{L-1} J_{L-2} \cdots J_l.$$

若这些 Jacobian 的典型奇异值持续小于 1, 梯度指数衰减; 持续大于 1 则爆炸。Residual block

$$h_{l+1} = h_l + F_l(h_l)$$

把局部 Jacobian 变成

$$I + J_{F_l},$$

因此始终保留恒等通路。

9.9 Activation checkpointing: 用计算换内存

朴素 reverse mode 会保存大量中间 activation。Checkpointing 只保存一部分节点, 反向到某段时重新做前向以恢复缺失 activation。于是可用更多 FLOPs 换更少内存。

如果网络有 L 层, 最简单的分段 checkpointing 可把 activation memory 从 $O(L)$ 降到近似 $O(\sqrt{L})$, 代价是重算。LLM 训练中的“显存优化”本质上是计算图上的时空复杂度设计。

9.10 从“算梯度”到“梯度解释”: 样本之间会互相帮助还是干扰?

对样本 i 做一步

$$\theta' = \theta - \eta g_i, \quad g_i = \nabla L_i(\theta).$$

另一样本 j 的损失一阶展开:

$$L_j(\theta') \approx L_j(\theta) - \eta g_j^\top g_i.$$

因此:

$$g_i^\top g_j > 0 \Rightarrow \text{训练 } i \text{ 倾向帮助 } j,$$

$$g_i^\top g_j < 0 \Rightarrow \text{训练 } i \text{ 可能伤害 } j.$$

这把“数据之间的关系”从 embedding 相似度提升到 learning dynamics similarity。

9.11 Research Frontier I: Learning Dynamics of LLM Finetuning

Research Frontier | Learning Dynamics of LLM Finetuning

会议身份: ICLR 2025 Outstanding Paper。^a

Ren 与 Sutherland 研究 finetuning 时，一个训练 response 如何逐步改变其他潜在 response 的概率，并用统一的 step-wise decomposition 解释 instruction tuning 与 preference tuning 中的若干现象，包括某些 hallucination strengthening，以及 off-policy DPO 训练过久时连 preferred outputs 也被压低的 squeezing effect。

本章连接: 一次参数更新只是

$$\Delta\theta \approx -\eta \nabla_{\theta} L,$$

但某个回答 y' 的 log-probability 变化近似为

$$\Delta \log p_{\theta}(y' | x') \approx \nabla_{\theta} \log p_{\theta}(y' | x')^{\top} \Delta\theta.$$

于是 finetuning 的行为后果可以被还原为梯度/score function 的相互作用，而不只看最终 benchmark。

^a论文: [ICLR Proceedings](#).

9.12 Research Frontier II: 早期梯度如何显露语义关联?

Research Frontier | How Do Transformers Learn to Associate Tokens: Gradient Leading Terms Bring Mechanistic Interpretability

会议身份: ICLR 2026 Oral。^a

该工作分析 attention-based language model 的早期训练动力学，通过 gradient leading-term approximation 推导权重的闭式近似，并把学习到的语义关联分解为与语料统计相关的几个基础映射（如 bigram、token interchangeability 与 context mappings）。实验进一步检查这些理论近似与实际 LLM 权重之间的对应。

研究品味: mechanistic interpretability 不一定只能在训练结束后“解剖神经元”。另一条路线是从更新方程出发，问：训练最初几步的主导梯度项已经预言了哪些结构？这把 autodiff 从工程工具变成理论分析对象。

^a论文: [ICLR 2026 paper](#); Oral 日程: [ICLR 2026 Thursday](#).

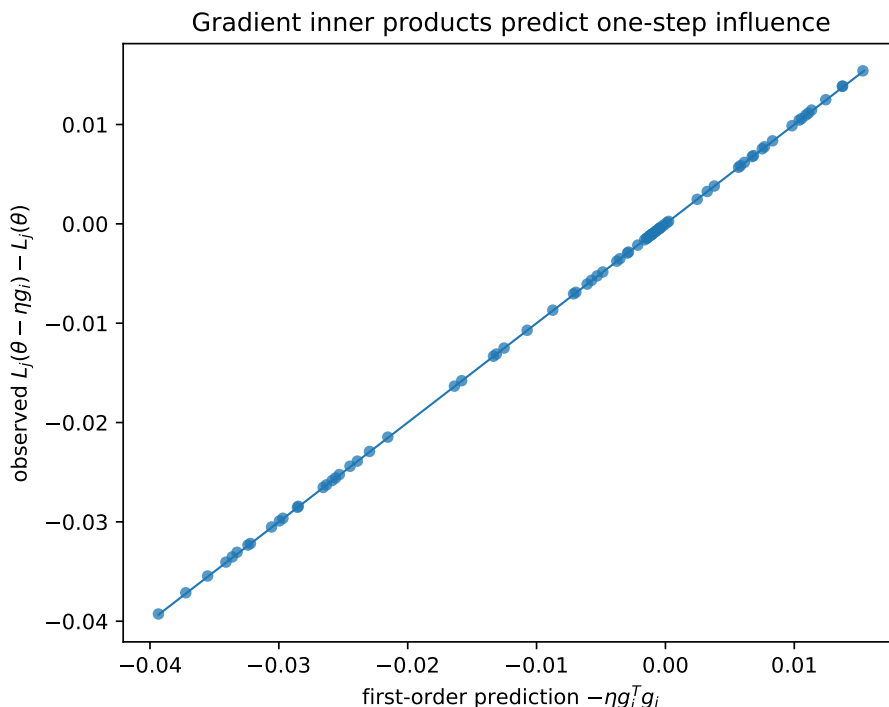


图 9.1 机制级验证：对一个样本做很小的一步更新后，其他样本 loss 的真实变化与 $-\eta g_i^\top g_j$ 的一阶预测高度一致。步长变大后偏离会增加，这正体现 Taylor 近似的边界。

Mini Lab | Frontier Replication 9: Gradient Influence 与一阶预测

运行 `labs/frontier/ch09_gradient_influence.py`。基础任务：比较预测 $-\eta g_i^\top g_j$ 与真实 ΔL_j 。进阶任务：逐步增大学习率，画出相关系数何时明显下降；再加入 HVP 二阶修正。研究任务：在一个 tiny language model 或字符模型上，把训练 examples 按 prompt/response 分组，比较 embedding similarity 与 gradient similarity 哪一个更能预测 finetuning transfer。

From Textbook to Research | 下一步：梯度算出来以后，怎样让 100 层、1000 层甚至数百亿参数模型稳定训练？

Backprop 只回答“梯度怎么算”，并不回答“更新是否稳定”“信号是否跨层保持尺度”“优化器状态能否放进显存”。下一章将把初始化、归一化、残差、momentum、AdamW、低秩 optimizer states 与 scaling 放到同一个动力系统视角下。

9.13 Deep Dive: Forward mode 与 Reverse mode 的成本对偶

若 $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$ ，计算完整 Jacobian：forward mode 约需 n 次方向传播，reverse mode 约需 m 次反向传播。因此：

- n 小、 m 大: forward mode 更自然;
- n 大、 m 小: reverse mode 更自然;
- 神经网络训练通常是数百万到数十亿参数 $\rightarrow$ 一个标量 loss, 因此 reverse mode 占主导。

但在 Jacobian regularization、implicit layer、meta-learning 中, JVP 与 VJP 往往会组合使用。

9.14 Deep Dive: Pearlmutter Trick 的思路

要计算 Hv , 定义

$$g(\theta) = \nabla L(\theta), \quad \phi(\theta) = g(\theta)^\top v.$$

则

$$\nabla \phi(\theta) = H^\top v = Hv$$

(Hessian 对称时)。因此二阶方向信息可以通过“对梯度与向量的内积再求一次梯度”得到。现代 autodiff 框架会自动完成这类 nested differentiation。

9.15 Deep Dive: 一阶 influence 何时失效?

Taylor 展开下一项为

$$L_j(\theta - \eta g_i) = L_j(\theta) - \eta g_j^\top g_i + \frac{\eta^2}{2} g_i^\top H_j g_i + O(\eta^3).$$

当步长、曲率或梯度范数变大时, 二阶项不可忽略。LLM finetuning 里若连续做成千上万步, 不能把单步一阶影响简单线性相加而不检查路径变化。

分层训练 | 本章分层训练

A | 手算: 为 $L = (wx - y)^2/2$ 画计算图并逐节点反传。

B | 推导: 证明线性层权重梯度为外积 $\delta x^\top$ 。

B | 编程: 实现一个只支持加法、乘法、tanh 的 scalar autodiff engine, 并用 finite difference 检查。

C | 研究: 用 HVP 给 gradient influence 加二阶修正, 量化在哪个学习率区间明显优于一阶式。

9.16 Proof & Worked Example Clinic

Dependency Check | 先修与跨章连接

自动微分是从第 10 章训练稳定到第 18 章 RLVR 的共同计算底座。

Worked Example | 反向模式的计算账本

若计算图有 M 个标量中间量、最终只有一个标量 loss，正向模式对每个参数方向分别传播 tangent，成本随输入维数增长；反向模式从 loss 传播 adjoint，一次 backward 即得到全部参数梯度。命题 9.1 因此不是“链式法则换个名字”，而是选择了最适合 many-input/one-output 的计算方向。

Proof Clinic | 一个两层网络手算 backward

设 $h = Wx$, $a = \tanh h$, $L = \frac{1}{2}(v^T a - y)^2$ 。记 $e = v^T a - y$ ，则

$$\frac{\partial L}{\partial v} = ea, \quad \frac{\partial L}{\partial h} = ev \odot (1 - a^2), \quad \frac{\partial L}{\partial W} = [ev \odot (1 - a^2)] x^T.$$

每一步都应写出 shape；shape mismatch 是最常见的反传推导错误。

Research Assignment | Gradient influence 可证伪实验

对 finetuning 数据做 leave-one-example influence 近似，比较真实删除重训的参数/输出变化。至少在小模型上报告相关性与失败点，并比较一阶 gradient dot-product 与 Hessian-preconditioned influence。

最低交付：1 页假设与指标声明；可复现代码与固定 seed；主结果图；至少一个失败/反例；200–400 字结论边界。**禁止：**只报告最好一次 run、只给截图、把 mechanism-level reproduction 表述为 paper-scale replication。

研究阅读与复现检查表

1. finetuning dynamics 的 step-wise decomposition 使用了哪些 Taylor/局部近似？
2. 单步 influence 如何累积到多步？梯度方向随参数变化会造成什么误差？
3. “gradient leading term” 在训练的哪个时间尺度内有效？后期 feature learning 会破坏哪些闭式近似？
4. 梯度相似度与表示相似度分别回答什么问题？二者何时可能完全不同？
5. 对数十亿参数模型，per-example gradient 内积怎样用低秩、随机投影、NTK 或 Fisher 近似？

习题

1. 对 $y = \sin(ax + b)$ 手工分别做符号微分、有限差分与计算图反传，比较三种方法。
2. 推导两层 MLP 的 W_1, W_2, b_1, b_2 梯度并检查维度。
3. 证明一个变量被计算图多个分支使用时，其 adjoint 必须累加各分支贡献。
4. 解释 JVP 与 VJP 的维度，并分别给出一个适用场景。
5. 推导 softmax + cross-entropy 合并后的 VJP，说明为何无需构造 $K \times K$ Jacobian。
6. 写程序对随机 MLP 做 central-difference gradient check，扫描 $h = 10^{-1}$ 到 10^{-8} 。
7. 推导 residual chain 两层时的 Jacobian $(I + J_2)(I + J_1)$ ，指出恒等短路径对应哪些项。
8. 使用 autodiff 实现 Hessian-vector product，并用显式 Hessian 的小模型结果验证。
9. 从 Taylor 展开推导 $\Delta L_j \approx -\eta g_i^\top g_j$ ，再写出二阶修正。
10. 设计实验比较 embedding cosine、gradient cosine 与实际 one-step transfer 三者的相关性。